

Class06: R functions

Shawn Xiao (PID:A18276668)

All functions in R have at least 3 things:

- A **name**, we pick this and use it to call the function.
- Input **arguments**, there can be multiple comma separated inputs to the function.
- The **body**, lines of R code that do the work of the function.

Our first wee function:

```
add <- function(x){  
  x+1  
}
```

Let's test our function

```
add(10)
```

```
[1] 11
```

```
add12 <- function(x,y=1){  
  x+y  
}
```

```
add12(10)
```

```
[1] 11
```

```
add12(10,10)
```

```
[1] 20
```

A second function

Let's try something more interesting. Make a sequence generation tool

The `sample()` function could be useful here

```
sample(1:10, size=3)
```

```
[1] 7 2 8
```

```
sample(c("A","T","G","C"),size=5,replace=TRUE)
```

```
[1] "G" "G" "A" "A" "C"
```

Turn this snippet into a function that returns a user specified length DNA sequence, let's call it `generate_dna`

```
generate_dna <- function(x=10){  
  v <- sample(c("A","T","G","C"),size=x,replace=TRUE)  
  cat("Well Done You!\n")  
  return(v)  
}
```

```
generate_dna(5)
```

```
Well Done You!
```

```
[1] "T" "T" "A" "T" "G"
```

```
s <- generate_dna(15)
```

```
Well Done You!
```

```
s
```

```
[1] "A" "C" "C" "A" "A" "C" "G" "C" "T" "T" "A" "G" "G" "G" "A"
```

I want the option to return a single element vector like "GGAGTAC"

```
paste(s,collapse="")
```

```
[1] "ACCAACGCTTAGGGA"
```

```
generate_dna <- function(x=10, fasta=FALSE){  
  v <- sample(c("A","T","G","C"),size=x,replace=TRUE)  
  
  cat("Well Done You!\n")  
  
  # Make a single element vector  
  s <- paste(v,collapse="")  
  
  # Adding fasta logic  
  if(fasta){  
    return(s)  
  } else{  
    return(v)  
  }  
}
```

```
generate_dna()
```

Well Done You!

```
[1] "G" "T" "A" "C" "T" "T" "T" "G" "C" "C"
```

```
generate_dna(10, fasta=T)
```

Well Done You!

```
[1] "CCCAATGGTC"
```

```
generate_prot <- function(ami=10, fasta=TRUE){  
  a <- c("A","R","N","D","C","Q","E","G","H","I","L","K","M","F","P","S","T","W","Y","V")  
  p <- sample(a,size=ami,replace=TRUE)  
  
  # Make a single element vector
```

```

t <- paste(p,collapse="")

# Adding fasta logic
if(fasta){
  return(t)
} else{
  return(p)
}
}

```

```
generate_prot(20)
```

```
[1] "DFYQLDVTTCGLPMIPFALK"
```

Q. Generate random protein sequences between lengths 5 and 12 amino acids.

One approach is to do this by brute force calling our function for each length 5 to 12

Another approach is to write a `for()` loop to iterate over the input values 5 to 12

A very useful third R specific approach is to use the `sapply()` function.

```

seq_length <- 5:12
for(i in seq_length){
  cat(">",i,"\n")
  cat(generate_prot(i))
  cat("\n")
}

```

```

> 5
TLRVF
> 6
MQITFC
> 7
CEFNYLN
> 8
PPNNDALD
> 9
MLNKYGEWA
> 10
WFPAAYTEPQ
> 11

```

```
QQGKFRHNCI  
> 12  
SWCMYNVVCWKP
```

```
sapply(5:12,generate_prot)
```

```
[1] "HFPCW"      "WANPGV"      "CDSEMPM"      "VLQASYLQ"      "FRVFARFID"  
[6] "AYWPNRRDNK" "HQGPSKHGISM" "MKLGMWHCKTLK"
```

Key Point: Writing functions in R is doable but not the easiest thing in the world. Starting with a working snippet of code and then using LLM tools to improve and generate additive functions to make it more useful.